Planetary motion simulator

Joona Levä
018215159

December 2025

Contents

1 Introduction

The purpose of the project was to create a program capable of simulating planetary systems using the programming language Fortran. The goal was to accurately portray the motion of the solar system. The scope of the simulation was limited to Newtonian gravity, which was numerically iterated using the velocity Verlet algorithm. Initial positions and velocities of individual objects were calculated from their real orbital parameters. Results of the simulation were visualized in a 3D-animation.

2 Methods

The simulation model is limited to Newtonian motion, and the only force accounted for is gravity. Objects are treated as collisionless point masses. Therefore, the accuracy of the simulation is limited to long distance interactions. Motion is represented in a 3-dimensional vector space using the Cartesian coordinate system.

Gravitational forces are calculated using Newton's law of gravitation

$$\vec{F} = G\frac{m_1 \cdot m_2}{r^2}\hat{r}, \tag{1}$$

where m_1 and m_2 are the masses of the objects, G is the Newtonian gravitational constant, and r is the distance between the objects. Distance is calculated from the 3-dimensional position vectors $\vec{x}$ with

$$\vec{r} = \vec{x}_1 - \vec{x}_2 \tag{2}$$

and the accelerations of the objects are calculated using Newton's second law of motion

$$\vec{F} = m\vec{a}. \tag{3}$$

Positions and velocities are numerically integrated from the acceleration using Verlet integration; more specifically, the velocity Verlet algorithm. The algorithm is as follows when time is $t = i\Delta t$ and $i \geq 0$ is the index of iteration:

$$\vec{x}_{i+1} = \vec{x}_i + \vec{v}_i\Delta t + \frac{1}{2}\vec{a}_i\Delta t^2 \tag{4}$$

$$\vec{a}_{i+1} = \frac{\vec{F}(\vec{x}_{i+1})}{m} \tag{5}$$

$$\vec{v}_{i+1} = \vec{v}_i + \frac{1}{2}(\vec{a}_i + \vec{a}_{i+1})\Delta t \tag{6}$$

Total gravitational force is calculated as a sum of the forces exerted by each object:

$$\vec{F}_n(\vec{x}) = \sum_{k,k\neq n}^{N} G\frac{m_n \cdot m_k}{r_k^2}\hat{r}_k, \tag{7}$$

where n is the index of the current object, $\vec{r}_k = \vec{x}_k - \vec{x}_n$, and N is the total number of objects in the system.

The method was chosen because it is more stable for the total energy of the system than the Euler method and less computationally demanding compared to the fourth order Runge-Kutta method. Accuracy of the produced orbits is overall mostly dependent on the chosen length of the timestep.

2.1 Cartesian position and velocity from orbital parameters

There is an optional initialization program that uses real orbital parameters to produce the initial coordinates for the simulation. Input parameters for it are the semi-major axis a, eccentricity e, inclination i, longitude of the ascending node Ω, argument of periapsis ω, time of periapsis passage τ, and standard gravitational parameter $\mu = G(M + m)$. All the listed values can be easily found for most solar system bodies on their wikipedia pages.

Keplers method of calculating position as a function of time was utilized to solve position and velocity in perifocal coordinate system. Rotation matrices were then used to calculate the motion in cartesian coordinate system.

The method is built for two-body systems where M is the mass of the larger central object and m is the lighter orbiting object. It is, however, still an accurate approximation for multiple-body systems where the central object is much larger than the orbiting bodies, which is the case for our solar system.

The first step is to calculate the eccentric anomaly E numerically from the Kepler equation

$$E - e \sin E = M = n(t - \tau), \tag{8}$$

where M is the mean anomaly, t is the current time, and $n = \frac{\mu}{a^3}$ is the mean motion of the object. Mean motion is calculated with Keplers third law of planetary motion.

True anomaly f can now be calculated with equation

$$(1 - e) \tan^2 \frac{f}{2} = (1 + e) \tan^2 \frac{E}{2} \tag{9}$$

and heliocentric distance r with

$$r = a \left(1 - e \cos E\right). \tag{10}$$

Perifocal coordinates of the orbiting bodies can now be attained:

$$\begin{bmatrix} x_{\text{perifocal}} \\ y_{\text{perifocal}} \\ z_{\text{perifocal}} \end{bmatrix} = \begin{bmatrix} r \cos f \\ r \sin f \\ 0 \end{bmatrix}, \quad \begin{bmatrix} v_{\text{perifocal}} \\ v_{\text{perifocal}} \\ v_{\text{perifocal}} \end{bmatrix} = \begin{bmatrix} -\frac{\mu}{h} \sin f \\ \frac{\mu}{h} \left(e + \cos f\right) \\ 0 \end{bmatrix} \tag{11}$$

$h = \sqrt{\mu a (1 - e^2)}$ is the specific relative angular momentum of the orbiting body.

With the perifocal coordinates, true equatorial cartesian coordinates can be calculated using 3D rotation matrices:

$$\begin{bmatrix} x_{\text{equatorial}} \\ y_{\text{equatorial}} \\ z_{\text{equatorial}} \end{bmatrix} = R \begin{bmatrix} x_{\text{perifocal}} \\ y_{\text{perifocal}} \\ z_{\text{perifocal}} \end{bmatrix}, \tag{12}$$

where $R = R_z\left(\Omega\right) R_x\left(i\right) R_z\left(\omega\right)$ and

$$R_x\left(\theta\right) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}, \quad R_z\left(\theta\right) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{13}$$

3 Implementation of methods

The project is divided into three independent programs: two written in Python and one in Fortran. The simulation program itself is written in Fortran and it has two module dependencies: *vectors* and *constants*. The *vectors* module defines a derived type,

vector, to be used as mathematical 3D vectors. In addition, it also defines all necessary mathematical vector operations as functions. Operator overloading is utilized for simple vector operations, such as addition and subtraction, to improve readability.

The *constants* module contains all the necessary mathematical constants used in the program, as well as the kinds of integer and real numbers. The real kind is set by default to 8 bytes and integer kind to 4 bytes.

The main simulation program itself is divided into five distinct sections. First, it reads the user given simulation configuration file *config.txt*, which contains all the simulation parameters. These are the total number of objects N, index of the units used by the program, index of the central object, simulation duration, number of steps, interval of console status outputs, and interval of positional data saves. This section contains multiple error handling elements that provide specific error messages if the configuration parameters are incorrect. There is also a safety in place where the program asks the user to manually proceed if the expected file size exceeds one gigabyte.

Currently, there are three possible unit configurations for position, time, and mass. Index one is meters, seconds, and kilograms; two is astronomical units, days, and kilograms; and three is parsecs, years, and solar masses. Index of the central object can be any integer. If it is a number between one and N, it is the index of the planet to be placed into the origin. When the index is zero, the center of mass of the system is placed into the origin. Any other number results in no specific central object. If no central object is specified, the center of mass of the system is able to drift.

Second, the program reads the user provided input data file *input.dat*, which contains all the initial positions, velocities, and masses of the system bodies. Before the actual reading, arrays are allocated to the total object amount provided in the configuration file. This section also contains two error handling elements, which ensure that the input data file contains the correct amount of data points. The structures of the user files are detailed in section 3.1.

The third section is the initialization of the motion arrays. There are individual vector arrays for position, velocity, acceleration, and gravitational force. Position and velocity are set to the values provided in the input file. The gravitational force for each object is calculated using equation (7) with two do-loops. Finally, acceleration is calculated using Newton's second law of motion. Before beginning the simulation, the initial status is printed to the console using subroutine `sim_stat()`.

The fourth section is where the simulation is actually executed. A file is created for the output data used later in visualization. Then, in a single do construct, motion is iterated with the velocity verlet algorithm with the duration and time step length defined in the configuration file. Time and position coordinates of the objects are written into the file and printed to the console at the predefined intervals.

The fifth section contains all the subroutines and functions used in the program. These include the aforementioned `sim_stat()`, which prints the current simulation status to the console, as well as a function for equation (1), a subroutine for configuration file verification, and a subroutine for the centering of the given object. Centering is done by subtracting the positional coordinates of the center object from each object.

One of the Python programs, *init.py*, is used for the creation of the simulation input

files. It contains the simulation configurations and the object list. The object list contains names, orbital parameters, and masses of the objects. These are then used to calculate the semi-realistic initial position and velocity vectors using methods described in section 2.1. The program creates the simulation input files *config.txt* and *input.dat* from the given values, as well as a file *sequence.txt*, which is used by the visualization program to label objects correctly. Python library Numpy is used for math operations and library CSV for file writing.

The other Python program, *plot.py*, is used for the visualization of the simulation output data. It utilizes the Matplotlib library to display the objects in a 3D environment. In addition to the data file, *output.dat*, the program uses the file *config.txt* to display units correctly and the file *sequence.txt* to display object names correctly. The two latter files are not required for the program to function but provide additional visual appeal.

3.1 Data structure

The simulation configuration file, *config.txt*, should contain seven parameters each divided by a tab key. These parameters, in order, are the number of bodies in the simulation, index of used units, index of the object at origin, duration of the simulation, number of steps, step interval of console print-outs, and the step interval of position data saves into the output file.

The structure of the input data file, *input.dat*, is as follows:

$m_{n=1}$	m_2	m_3	...
x_1	x_2	x_3	...
y_1	y_2	y_3	...
z_1	z_2	z_3	...
$v_{x,1}$	$v_{x,2}$	$v_{x,3}$	...
$v_{y,1}$	$v_{y,2}$	$v_{y,3}$	...
$v_{z,1}$	$v_{z,2}$	$v_{z,3}$	...

n is the index of the object. Each object occupies one column, and the rows are in order from top to bottow: mass, position, and velocity. Position and velocity are divided into their cartesian components.

The structure of the simulation output data file, *output.dat*, is as follows:

$t_{i=1}$	$x_{i=1,n=1}$	$y_{1,1}$	$z_{1,1}$	$x_{1,2}$	$y_{1,2}$	$z_{1,2}$	...
t_2	$x_{2,1}$	$y_{2,1}$	$z_{2,1}$	$x_{2,2}$	$y_{2,2}$	$z_{2,2}$	...
t_3	$x_{3,1}$	$y_{3,1}$	$z_{3,1}$	$x_{3,2}$	$y_{3,2}$	$z_{3,2}$	...
...	...	...	...	...	...	...	...

i is the index of the saved iteration, and n is the index of the object. The first column contains the current time. After that, each object occupies three columns for each positional coordinate.

The object sequence file, *sequence.txt*, contains the order of the objects in the output data file. Each object name is on its own row.

3.2 How to compile and run the simulation

Compilation of the simulation was done in Linux using GFortran from the GNU Compiler Collection, which is built into most Linux distributions. The simulation should be able to be compiled using other operating systems and compilers, but only Linux and GFortran have been tested. All following commands require that the folder, in which the accessed files are located in, is opened in the console.

When using Linux, compiling can be done with the provided Make-script inside the file *Makefile*. To activate the script, open the source code folder in the console and type in the bash-command `make`. The script utilizes the aforementioned GFortran, and thus must be edited if another compiler is used. If the script does not work, compiling can be done manually with bash-command `gfortran -O3 -o simulation constants.f90 vectors.f90 simulation.f90`.

The simulation file can be run in Linux with bash-command `./simulation` if the two input files are in the same folder. It is heavily recommended, but not necessary, to use the initialization program *init.py* for the creation of the input data and configuration files. It also creates the target sequence file, which allows the data visualizer to display object names. The initialization program can be run with the command `python3 init.py`. The initialization program file can be edited with any text editor to change the simulation parameters as well as the object list.

The data visualization program, *plot.py*, can be run with the command `python3 plot.py` when the simulation output file *output.dat* is inside the folder.

4 Results

To test the accuracy of the simulation, several orbital periods were measured and compared to the real values.

To test the effects of timestep length, the orbital period of Jupiter was measured in a two-body system with six different timestep lengths. The results are in table 1. From the results, it is apparent that the timestep affects the rounding of the result more than the value. Noticeable differences in the results, in addition to the rounding, were observed only when the timestep length exceeded 100 days. Since Jupiter's true orbital period is about 4333 days, realistic values were achieved even at a timestep of ten days.

When the simulation duration was set to the true orbital period of 4333 days, Jupiter's orbit in simulation was 0.0062689 au short from a complete orbit. Timestep of 0.01 days was used for this. The result is verified by the value of complete orbital period, which was measured to be a little over the true value. The difference may be a result of the two-body approximation or the lack of accuracy in the used orbital parameters. With the same timestep, orbital period of the Sun was observed to be the exact same as Jupiter's. The average radius of motion for the Sun was calculated to be 0.0049689 au, which is accurate for the distance to the center of mass of the system.

Timestep h (day/step)	Jupiter orbital period (day)
0.01	4333.83
0.1	4333.8
1	4334
10	4330
100	4400
500	5500

Table 1: Orbital periods of Jupiter with different time steps in a Sun-Jupiter two-body system. Orbital parameters of Jupiter were collected from wikipedia.

The length of a year was estimated by measuring the orbital period of the Earth in a Sun-Earth two-body system. A month was measured in an Earth-Moon system. In both, a timestep of 0.001 days was used. Earth's orbital period was measured to be 365.256 days, which is the exact value of a sidereal year. Moon's orbital period was measured to be 27.291 days, which is only about 0.11% less than the correct sidereal month of 27.322 days. Differences are probably the result of the two-body approximation and inaccurate orbital parameters. However, the results are still very close.

Target	Timestep h (day/step)	Orbital period (day)
Earth	0.001	365.256
Moon	0.001	27.291
Mercury	0.001	87.936
Mercury	0.01	87.94
Mercury	0.1	87.9

Table 2: Other measured periods. Periods of Earth and the Moon were simulated in Earth-Sun and Moon-Earth two-body systems. Mercury was simulated in 9-body solar system which included all eight major planets. All orbital parameters were collected from wikipedia.

Finally, the orbital period of Mercury was measured using three different timesteps in a 9-body solar system featuring all eight major planets. It was again noted that the length of the timestep did not noticeably affect the results, apart from the rounding. The orbital period was measured to be 87.936 days, which is just 0.0375% less than the true period of 87.969 days. From figure 3, it can be seen that the orbit did not fully connect as it would in a two-body system. The difference in the period is probably mostly due to inaccuracies in the orbital parameters. Lack of general relativity might also cause minor deviations at these distances from the Sun.

Overall, the simulation is very accurate with differences in orbital periods mostly less than a tenth of a percentile when small enough timestep is used. The execution time of the simulation is also quite fast. A simulation of one million steps for a 9-body solar system takes around 1.85 seconds to execute on my laptop. A ten million step simulation for the same system takes 18.27 seconds. With 23 bodies execution time of a 30 million

step simulation took 2 minutes and 48 seconds. This simulation was able to capture the full orbital periods of outer dwarf planets like Pluto and Eris, while keeping the inner solar system orbits realistic.

5 Conclusions

In summary, the simulation utilized velocity Verlet algorithm for numerical integration of Newtonian gravity. Initial positions and velocities were calculated using Keplers method for position as a function of time. The simulation returned accurate orbital periods that differed only around tenth of a percentile from the real values when using appropriate timestep lengths. Length of the timestep was observed to have a very limited effect on the results, apart from the rounding of the values.

Significance of the effects of timestep length could be better observed by interpolating the exact time a full orbit is reached. Currently, the step closest to a full orbit was chosen to represent the period. This step was usually significantly over or behind the full rotation and thus did not represent properly the true orbital period corresponding to the used timestep length.

The accuracy of the simulation itself could be further improved by using a more accurate numerical integration methods such as the fourth-order Runge-Kutta method. General relativity could also be accounted for to improve accuracy when simulating objects of greater mass than our Sun.

The current source code for the simulation might also be able to be optimized further for improved execution times, as well as reduced output file sizes.

References

https://en.wikipedia.org/wiki/Kepler%27s_laws_of_planetary_motion#Position_as_a_function_of_tin
https://en.wikipedia.org/wiki/Verlet_integration#Velocity_Verlet
https://en.wikipedia.org/wiki/Newtoński_law_of_universal_gravitation
https://en.wikipedia.org/wiki/Perifocal_coordinate_system#Conversion_between_coordinate_systems

Appendices

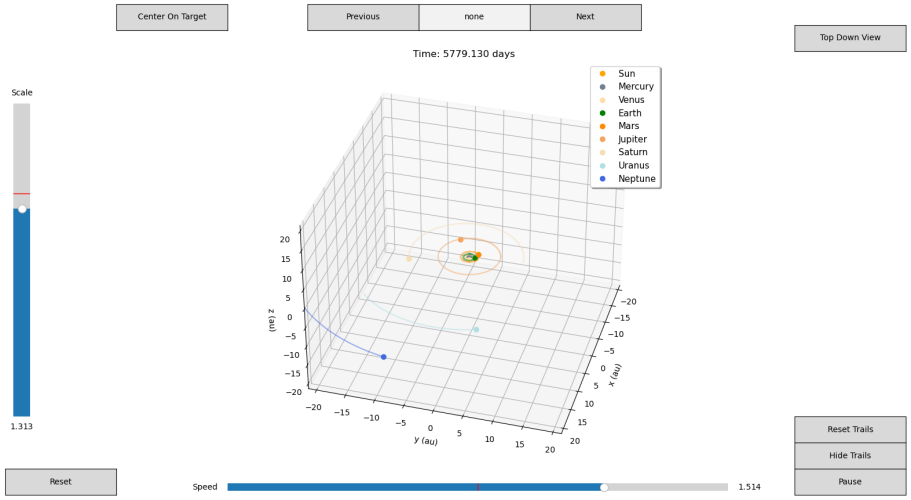

Figure 1: Simulated solar system with the animation interface visible.

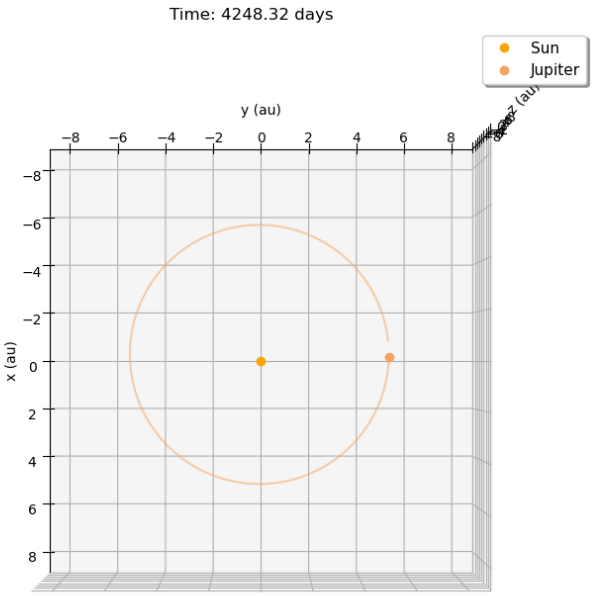

Figure 2: Sun-Jupiter system.

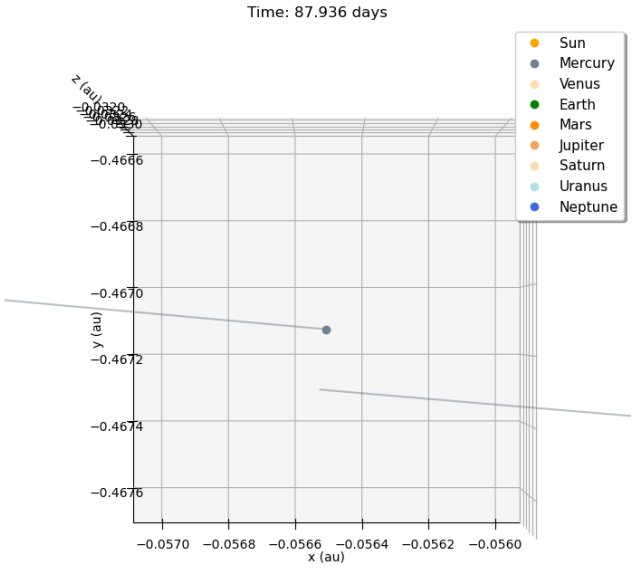

Figure 3: The orbit of Mercury does not connect in a full solar system simulation.